

QueryMind

Natural Language to SQL • Full-Stack Platform

Comprehensive Project Report

Generated: April 10, 2026

Angular 18

Django 5

PostgreSQL 16

Claude AI

Docker

AWS

Table of Contents

1. Executive Summary
2. Technology Stack
3. System Architecture
4. Backend: Django REST API
5. Database Design
6. NL2SQL AI Engine
7. Frontend: Angular 18 SPA
8. Analytics & OLAP
9. Transaction Management
10. API Reference
11. DevOps & Deployment
12. Documentation & Phases
13. Project Statistics

1. Executive Summary

QueryMind is a production-grade, full-stack Natural Language to SQL (NL2SQL) platform built around a University Management System demo database. It enables users to query a relational database using plain English, powered by Anthropic's Claude AI. The platform is designed as a comprehensive DBMS portfolio project that demonstrates mastery across the entire database syllabus: ER modeling, relational conversion, normalization (BCNF), SQL, joins, indexing, ACID transactions, OLAP analytics, ETL pipelines, and an AI-powered natural language interface.

The system features an animated Angular 18 frontend with Three.js particle effects, GSAP transitions, D3.js OLAP visualizations, and a Monaco SQL editor; a Django 5 REST API backend with schema-aware query generation, SQL validation, and rate-limited execution; and a PostgreSQL 16 database with both normalized OLTP and denormalized star-schema analytics layers.

Feature	Description
Natural Language Queries	Ask questions in plain English; get validated SQL and live results instantly.
Schema-Aware AI	Claude AI receives full database schema context for accurate SQL generation.
Interactive ER Diagrams	Canvas-based entity-relationship diagrams with drag, zoom, and domain filtering.
OLAP Analytics Dashboard	D3.js visualizations with ROLLUP, CUBE, and GROUPING SETS analytics.
ACID Transactions	Serializable isolation for enrollment, grading, and payment workflows.
Production Deployment	Docker Compose locally; AWS ECS Fargate, RDS, S3, CloudFront in production.

2. Technology Stack

2.1 Frontend

Library	Version	Purpose
Angular	18.2	Core SPA framework with standalone components
TypeScript	5.5.4	Type-safe application logic
Three.js	0.183.2	WebGL particle background animations
GSAP	3.14.2	Timeline-based UI transitions and animations
D3.js	7.9.0	Custom SVG charts for OLAP analytics
Monaco Editor	0.55.1	VS Code-quality SQL display and editing
RxJS	7.8.1	Reactive streams and SSE handling

2.2 Backend

Technology	Version	Purpose
Python	3.11+	Runtime environment
Django	5.1.5	Web framework
Django REST Framework	3.15.2	RESTful API layer
psycopg	3.2.13	PostgreSQL async driver (binary)
Anthropic SDK	0.49.0	Claude AI integration for NL2SQL
sqlparse	0.5.3	SQL parsing and safety validation
Gunicorn	23.0.0	Production WSGI server
Sentry SDK	2.19.2	Error tracking and monitoring
SQLAlchemy	2.0.36	ORM for seed data generation
Faker	33.3.1	Realistic test data generation

2.3 Infrastructure

Technology	Version	Purpose
PostgreSQL	16	Primary OLTP + OLAP database
Docker Compose	Latest	Local development orchestration
Nginx	1.27	Reverse proxy and SPA serving
AWS ECS Fargate	-	Serverless container hosting

Technology	Version	Purpose
AWS RDS	-	Managed PostgreSQL
AWS S3 + CloudFront	-	Static frontend hosting + CDN
GitHub Actions	-	CI/CD pipelines

3. System Architecture

QueryMind follows a classic three-tier architecture with a clear separation between the presentation layer (Angular SPA), application layer (Django REST API), and data layer (PostgreSQL). Communication flows through a RESTful API with JSON payloads, supplemented by Server-Sent Events (SSE) for real-time token streaming during AI query generation.

3.1 Project Directory Structure

Directory	Purpose
.github/workflows/	CI/CD pipeline definitions (ci.yml, deploy.yml)
backend/core/	Core Django app: models, views, services, NL2SQL engine
backend/querymind_api/	Django project settings (base, local, production)
backend/sql/ddl/	Database DDL scripts (schema, views, indexes)
backend/sql/warehouse/	Star schema and OLAP query definitions
backend/sql/seeds/	Data generation scripts (SQLAlchemy + Faker)
frontend/src/app/	Angular standalone AppComponent (ts, html, css)
frontend/src/assets/	Static assets
frontend/src/environments/	Angular environment configurations
infrastructure/	AWS setup scripts and IAM policies
docs/	8-phase DBMS documentation
db/	Docker endpoint SQL initialization files

3.2 Data Flow

- Step 1. User types a natural language question in the Angular frontend.
- Step 2. Frontend sends POST /api/query with the question; SSE stream opens for thinking overlay.
- Step 3. Django view invokes the NL2SQL Agent, which serializes the live database schema.
- Step 4. Agent retrieves similar past queries for few-shot prompting context.
- Step 5. Claude AI (claude-sonnet-4-20250514) generates SQL with schema awareness.
- Step 6. sqlparse validates the SQL; destructive statements are rejected.
- Step 7. Validated SQL executes in a read-only PostgreSQL transaction.
- Step 8. Results, explanation, and metadata return to the frontend as JSON.
- Step 9. Frontend renders results in a data table, Monaco editor, and optional chart.
- Step 10. Query is logged to QueryLog for history, metrics, and future few-shot retrieval.

4. Backend: Django REST API

4.1 Django Models

The backend uses two lightweight Django models for operational tracking, while the rich university schema lives directly in PostgreSQL (managed via raw DDL scripts).

Model	Description
QueryLog	Stores every NL query attempt: input text, generated SQL, execution status, timing, row count, explanation, and error details.
EtlLog	Tracks analytics ETL pipeline runs: status, records loaded, start/finish timestamps, and error messages.

4.2 Core Services

Function	Description
get_schema_payload()	Returns tables, columns, constraints, relationships, and insertable tables for the frontend schema browser.
get_table_intelligence()	Per-table analytics: row count, null rates, indexes, sample rows, and suggested queries.
get_relationship_preview()	Sample joined rows for a foreign key relationship (interactive ER preview).
insert_record()	Safely inserts into whitelisted tables with input validation.
execute_demo_sql()	Deterministic demo query execution (no AI involved).
generate_demo_query()	Heuristic local SQL generator as fallback when AI is unavailable.
extract_limit()	Parses row limits from natural language ('top 10' -> 10, 'twenty five' -> 25).

4.3 Configuration

Django settings are split across **base.py** (shared), **local.py** (development), and **production.py** (AWS). Key configuration includes:

- PostgreSQL connection via psycopg3 with connection pooling
- CORS whitelist: localhost:4200, localhost:80, 127.0.0.1
- REST Framework: JSON-only renderer, custom rate throttling
- Sentry SDK integration for production error tracking
- No authentication layer (open API with rate limiting for portfolio demo)
- Environment variables for secrets (DJANGO_SECRET_KEY, ANTHROPIC_API_KEY)

5. Database Design

5.1 OLTP Schema (20 Entities)

The normalized OLTP schema models a complete university management system with entities spanning academic, financial, and housing domains. The design includes ISA hierarchies (Person -> Student/Faculty/Staff, Assessment -> Exam/Assignment), weak entities with composite keys, and comprehensive constraints.

Entity	Est. Rows	Kind	Description
Department	40	Master	Academic unit with budget, office, and contact info. PK: department_id.
Person	~1,000	Supertype	Base entity with ISA hierarchy. Type discriminator: person_type (student/faculty/staff).
Student	~600	Entity	Subtype of Person. Roll number, CGPA, program level, admission status.
Faculty	~80	Entity	Subtype of Person. Employee code, designation, salary, specialization.
Staff	~100	Entity	Subtype of Person. Job title, department assignment (nullable for central admin).
Semester	~20	Master	Academic periods: Monsoon, Winter, Summer terms with date ranges.
Course	~60	Entity	Courses with credits (1-6), levels (100-900), types (core/elective/lab/seminar).
Course_Prerequisite	-	Weak	Composite PK (course_id, prerequisite_course_id). Self-ref prevented.
Section	~200	Transaction	Course offering per semester. Capacity, delivery mode, schedule pattern.
Enrollment	~2,000	Transaction	Student-section registration. Grades, attendance, status tracking.
Assessment	~600	Weak	Base for Exam/Assignment ISA. Type, max marks, weightage percentage.
Exam	-	Subtype	Mode (offline/online), date, duration, hall assignment.
Assignment	-	Subtype	Release date, submission mode, max attempts.
Exam_Result	-	Weak	Composite PK (exam_id, student_id). Marks, grade letter, remarks.
Assignment_Submission	-	Weak	Composite PK (assignment_id, student_id, attempt_no). Score, feedback.
Fee_Invoice	~3,000	Transaction	Per-student per-semester billing. Tuition, hostel, scholarship amounts.
Fee_Payment	~3,000	Transaction	Payment records. Method (UPI/card/netbanking/cash), status tracking.
Hostel	~10	Master	Residential facilities. Type (boys/girls/mixed), capacity, warden.
Hostel_Room	~200	Weak	Rooms within hostels. Floor, bed capacity, room type.
Hostel_Allotment	~800	Transaction	Student room assignments per semester with date ranges.

5.2 Constraints & Integrity

- Foreign Keys:** ON DELETE CASCADE for ISA subtypes, ON DELETE RESTRICT for business references.
- CHECK Constraints:** Enum validation (person_type, delivery_mode, payment_method), numeric ranges (credits 1-6, level 100-900).
- UNIQUE Constraints:** Business keys (roll_number, employee_code, course_code, invoice UUID, transaction_ref).
- Date Validation:** end_date > start_date enforced on semesters and allotments.

- **Composite Keys:** Weak entities use multi-column primary keys (exam_id + student_id, etc.).

5.3 Database Views

View Name	Description
student_transcript	Full enrollment history: student info, course, semester, grades, attendance percentage.
faculty_load	Sections taught per faculty per semester with total capacity.
department_summary	Aggregate department stats: student count, faculty count, course count, average CGPA.

5.4 Analytics Star Schema

A denormalized star schema in the **analytics** namespace supports OLAP workloads without impacting OLTP performance.

Table	Type	Key Columns
dim_department	Dimension	department_id, code, name
dim_student	Dimension	student_id, roll_number, full_name, department, program_level, hostel_flag
dim_faculty	Dimension	faculty_id, employee_code, full_name, designation, department
dim_course	Dimension	course_id, code, title, credits, type
dim_time	Dimension	semester_id, academic_year, term_name, start/end dates
fact_enrollments	Fact	enrollment_id, dimension keys, grade_points, fees_paid, attendance_pct

6. NL2SQL AI Engine

The NL2SQL engine is the core intelligence of QueryMind, translating natural language questions into validated, safe SQL queries using Anthropic's Claude AI.

6.1 Architecture (NL2SQLAgent)

Stage	Description
Schema Serialization	Introspects PostgreSQL information_schema to build a live snapshot of all tables, columns, constraints, foreign keys, and
Few-Shot Retrieval	Searches past successful queries using sequence matching to find similar questions. These are injected into the prompt a
AI Generation	Sends the schema context, similar examples, and user question to Claude (claude-sonnet-4-20250514, 800 max tokens).
SQL Validation	Uses sqlparse to verify SQL structure. Rejects destructive statements: DROP, TRUNCATE, ALTER, INSERT, UPDATE, D
Safe Execution	Runs validated SQL in a read-only PostgreSQL transaction path.
Retry Logic	If execution fails, the error message is fed back to Claude for auto-correction (configurable retry count).

6.2 Safety Measures

- SQL parsed and validated via sqlparse before any execution
- Destructive keywords (DROP, TRUNCATE, ALTER) are blocked at the validation layer
- DELETE/UPDATE without WHERE clause rejected
- Read-only transaction mode for all AI-generated queries
- Rate throttling: 10 queries per minute per IP address
- Query timeout limits to prevent long-running operations
- No authentication bypass — open API by design with strict validation

6.3 Streaming & UX

The frontend subscribes to an SSE (Server-Sent Events) endpoint at `/api/query/stream` to receive token-by-token updates during AI generation. This powers a 3-stage thinking overlay animation: Schema Analysis -> SQL Generation -> Execution. Tokens are throttled at 40ms intervals for smooth visual feedback.

7. Frontend: Angular 18 SPA

7.1 Application Tabs

Tab	Features
Query	Natural language input, example chips, chat thread, thinking overlay, results table, Monaco SQL editor, chart visualization, EXPLAIN
History	Paginated query logs with status filtering (all/success/failed). Click to re-hydrate a past query.
Analytics	D3.js OLAP dashboard: department bar chart, enrollment trend area chart, course mix donut chart, query activity heatmap.
ER Diagram	Canvas-based interactive entity-relationship diagrams with domain filtering (academic/finance/housing/all), drag nodes, zoom/pan, r
Schema Browser	Searchable table list with column details, constraint badges, domain/entity classification. Intelligence panel (row counts, null rates, in
Settings	Theme selection (dark/light/oled/sepia), motion reduction toggle, tour reset, DDL viewer.

7.2 Visual Design

- **Theme:** Modern dark UI with electric blue accents and neon pink highlights. Four theme options: dark, light, OLED, and sepia.
- **Landing Page:** Hero section with Three.js particle canvas background, gradient text, and feature strip cards.
- **Transitions:** GSAP-driven tab switches, launch wipe animation, chat scroll, and thinking overlay stage progression.
- **Custom Cursor:** Halo effect that follows mouse movement across the application.
- **Responsive:** CSS Grid and Flexbox layout with reduced-motion support (prefers-reduced-motion media query).
- **Toast System:** Top-right notification toasts with auto-dismiss for user feedback.

7.3 Key Capabilities

- **Voice Input:** Web Speech API integration for dictating queries via microphone.
- **Text-to-Speech:** Speak query explanations aloud using the SpeechSynthesis API.
- **Command Palette:** Cmd+K shortcut opens a command palette for quick navigation.
- **Keyboard Shortcuts:** Cmd+Enter to submit, Escape to close modals, full keyboard accessibility.
- **Share Links:** Generate shareable URLs for specific query results.
- **CSV Export:** One-click export of query results to CSV files.
- **97 Example Queries:** Predefined examples grouped by topic (students, faculty, courses, finance, hostel, advanced).

8. Analytics & OLAP

QueryMind includes a full analytics layer demonstrating advanced SQL aggregation techniques against the star schema warehouse.

Query	Description
ROLLUP Query	Hierarchical grade point aggregation: department -> semester -> grand total. Uses GROUPING() function for level detection.
Department Performance	Per-department metrics: enrollment count, average grade points, attendance rates.
Term Trends	Semester-over-semester enrollment and grade trends as time-series data.
Faculty Impact	Faculty contribution analysis: enrollment counts and grade distributions.
Course Mix	Enrollment distribution by course type (core/elective/lab/seminar).
Query Activity	Hourly distribution of API query usage from QueryLog timestamps.

8.1 ETL Pipeline

An incremental ETL pipeline (`run_incremental_etl()`) loads new enrollments into the analytics star schema. It operates with `SERIALIZABLE` isolation, performs dimension upserts (department, time, course, faculty, student), loads fact rows, and tracks the last processed enrollment ID for incremental runs. Each run is logged to the `EtlLog` model with status, record counts, and timing.

9. Transaction Management

QueryMind demonstrates ACID transaction management with three carefully designed transactional workflows, each using appropriate PostgreSQL isolation levels.

Function	Isolation Level	Behavior
enroll_student()	SERIALIZABLE	Checks section capacity with SELECT FOR UPDATE, inserts enrollment record, creates fee invoice.
submit_grade()	REPEATABLE READ	Upserts exam result, updates enrollment grade fields. Consistent read snapshot prevents phantom reads.
record_fee_payment()	SERIALIZABLE	Validates outstanding balance, records payment, updates invoice status. Full serializability prevents lost updates.

All transactions use atomic blocks with appropriate isolation. The enrollment transaction uses SELECT FOR UPDATE to acquire row-level locks on sections, preventing concurrent over-enrollment. The payment transaction uses full serializability to prevent race conditions where two payments could exceed the outstanding balance.

10. API Reference

Method	Endpoint	Description
GET	/api/	API index with available endpoints
GET	/api/health	Health check / smoke test
GET	/api/schema	Compact schema for frontend (tables, columns, constraints, relationships)
GET	/api/schema/intelligence?table_name=X	Table stats: row count, null rates, indexes, samples, query suggestions
GET	/api/schema/relationship-preview?...	Sample joined rows for a foreign key relationship
POST	/api/query	Generate and execute NL query (rate limited: 10/min)
GET	/api/query/stream	SSE stream of token generation for thinking overlay
POST	/api/query/explain	EXPLAIN (FORMAT JSON) for a SQL string
POST	/api/query/fixit	Auto-repair failing SQL using Claude
GET	/api/history	Paginated query history (10 per page)
GET	/api/history/<id>	Single query log detail (share link hydration)
GET	/api/metrics	Aggregated metrics: total queries, success rate, latency stats
GET	/api/analytics/rollup	OLAP rollup data for dashboard charts
POST	/api/admin/etl/run	Trigger incremental ETL pipeline
POST	/api/records/insert	Insert record into whitelisted table

10.1 Rate Limiting

A custom **QueryRateThrottle** (extending DRF's SimpleRateThrottle) limits query generation to **10 requests per minute per IP address**. This is cache-based and applies only to the /api/query endpoint to prevent API abuse while keeping other endpoints freely accessible.

10.2 Serializers

Serializer	Description
QueryRequestSerializer	Accepts natural_language (required) and prior_sql (optional, for query refinement).
InsertRecordSerializer	Accepts table_name and record dict for safe data insertion.
QueryLogSerializer	Full query history with all metadata fields.
EtlLogSerializer	ETL run metadata with timing and status.

11. DevOps & Deployment

11.1 Docker Compose (Local)

Service	Image/Build	Details
postgres	postgres:16	PostgreSQL database with persistent volume. Health check via pg_isready. Init scripts from /db/sql/.
django	./backend	Django API with Gunicorn (2 workers, 4 threads) on port 8000. Depends on postgres health.
nginx	./frontend	Multi-stage build: Node 20 (Angular build) + Nginx 1.27 (serve). Proxies /api/ to django:8000. Port 80.

11.2 CI/CD Pipelines

CI Pipeline (ci.yml) — Runs on PR and push to main:

- Backend: Python 3.11, pip install, django check
- Frontend: Node 20, npm ci, Angular production build
- Docker: Validate docker-compose config

Deploy Pipeline (deploy.yml) — Push to main or manual dispatch:

- Assume AWS IAM role via OIDC
- Build and push backend image to ECR
- Build frontend and sync to S3 bucket
- Invalidate CloudFront distribution cache
- Force ECS Fargate service redeployment

11.3 AWS Production Architecture

Service	Purpose
ECS Fargate	Serverless container hosting for Django API
RDS PostgreSQL	Managed database with automated backups
S3	Static hosting for Angular build artifacts
CloudFront	CDN for frontend with edge caching
Route 53	DNS management and domain routing
Secrets Manager	Production secrets (Django key, Anthropic key, DB password)
CloudWatch	Logging and monitoring
ECR	Docker image registry for backend

11.4 Environment Variables

Variable	Description
DJANGO_SECRET_KEY	Django cryptographic signing key
DJANGO_DEBUG	Debug mode toggle (0 in production)
ANTHROPIC_API_KEY	Claude AI API key for NL2SQL
POSTGRES_DB	Database name (default: querymind)
POSTGRES_USER	Database user
POSTGRES_PASSWORD	Database password
POSTGRES_HOST	Database host (postgres in Docker, RDS endpoint in prod)
POSTGRES_PORT	Database port (default: 5432)
SENTRY_DSN	Sentry error tracking endpoint

12. Documentation & DBMS Phases

The project documentation is organized into 8 phases covering the complete DBMS curriculum, available in the `/docs/` directory.

Phase	Title	Content
Phase 1	ER & Relational Design	ER diagram (20 entities, 2 ISA hierarchies), relational schema conversion, normalization audit (BCNF), DDL
Phase 2	Demo Queries	30+ curated SQL examples demonstrating joins, aggregations, subqueries, window functions, and common
Phase 3	Star Schema & OLAP	Fact/dimension design rationale, denormalization decisions, ROLLUP, CUBE, and GROUPING SETS exam
Phase 4	Transaction Management	ACID properties in context, isolation levels (SERIALIZABLE, REPEATABLE READ, READ COMMITTED), c
Phase 5	Agent & Learning Layer	NL2SQL architecture, Claude integration design, schema serialization strategy, few-shot learning from quer
Phase 6-7	API & Frontend	REST endpoint specifications, Angular component architecture, UI/UX decisions (GSAP, Three.js, D3, Mon
Phase 8	Resume & Interview	2-minute presentation script, resume bullet points, architectural decision rationale for interviews.

13. Project Statistics

Metric	Value
Database Tables	20 OLTP entities + 6 star schema tables
Database Views	3 (student_transcript, faculty_load, department_summary)
API Endpoints	15 RESTful endpoints
Example Queries	97 predefined NL query examples
Frontend Tabs	6 (Query, History, Analytics, ER Diagram, Schema, Settings)
Themes	4 (Dark, Light, OLED, Sepia)
AI Model	claude-sonnet-4-20250514 (800 max tokens)
Docker Services	3 (PostgreSQL, Django, Nginx)
CI/CD Pipelines	2 (CI validation, Production deploy)
AWS Services Used	8 (ECS, RDS, S3, CloudFront, Route 53, Secrets Manager, CloudWatch, ECR)
Est. Seed Data Rows	~8,000+ across all tables
OLAP Query Types	6 (ROLLUP, Department, Trend, Faculty, Course Mix, Activity)
Transaction Workflows	3 (Enrollment, Grading, Payment)
Documentation Phases	8 covering complete DBMS curriculum

13.1 Key Architectural Decisions

- **Single Angular Component:** Monolithic AppComponent (~1000 lines) for simplicity in a demo/portfolio context, avoiding over-engineering.
- **No Authentication:** Open API with rate limiting instead of JWT/sessions — appropriate for a portfolio showcase.
- **Dynamic Schema Introspection:** Live information_schema queries vs. hardcoded ORM models ensures AI always sees current state.
- **Few-Shot Learning:** Similarity-based query history retrieval improves AI accuracy over time without fine-tuning.
- **SSE Streaming:** Token-by-token thinking overlay with 40ms throttle for engaging visual feedback.
- **Separate Star Schema:** Dedicated analytics namespace prevents expensive OLAP joins from impacting OLTP performance.
- **GSAP over CSS Animations:** Timeline-based, interruptible transitions for complex animation sequences.
- **D3 over Chart.js:** Custom SVG rendering for fine-grained OLAP visualization control.
- **Monaco Editor:** VS Code-quality SQL display reinforces the 'serious tool' aesthetic.
- **Docker Compose Override:** Local dev credentials embedded; production uses AWS Secrets Manager.

This report was auto-generated from the QueryMind source code and project documentation.

Generated on April 10, 2026 at 13:19